

Práctica: Semilla de datos y consultas LINQ de alto rendimiento

Paso a paso — Poblar roles maestros con HasData, auditar cambios y optimizar con AsNoTracking

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 4 — Persistencia con Entity Framework Core
Docente	Edwin Barrientos
Duración estimada	60-90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Configurar datos maestros semilla (Seed Data) en EF Core para Roles y Centros de Costo iniciales.
- Implementar un interceptor o sobrecarga de SaveChangesAsync para poblar automáticamente los campos de EntidadAuditable.
- Escribir consultas LINQ con Include y ThenInclude evitando el antipatrón del N+1.
- Optimizar consultas de solo lectura mediante AsNoTracking().
- Construir proyecciones tipadas con Select() para transferir únicamente las columnas requeridas por el cliente.

2. Desarrollo paso a paso

Paso 1 — Semilla de datos maestros con HasData()

En RolConfiguration.cs y CentroCostoConfiguration.cs, definimos los datos base de la organización.

RolConfiguration.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using SolicitudesApp.Domain.Entidades;

namespace SolicitudesApp.Infraestructura.Persistencia.Configuraciones;

public class RolConfiguration : IEntityTypeConfiguration<Rol>
{
    public void Configure(EntityTypeBuilder<Rol> builder)
    {
        builder.ToTable("Roles");
        builder.HasKey(r => r.Id);
        builder.Property(r => r.Nombre).IsRequired().HasMaxLength(50);

        builder.HasData(
            new Rol { Id = 1, Nombre = "Usuario" },
            new Rol { Id = 2, Nombre = "Revisor" },
            new Rol { Id = 3, Nombre = "Aprobador" },
            new Rol { Id = 4, Nombre = "Tesoreria" }
        );
    }
}
```

Paso 2 — Automatizar la auditoría en SaveChangesAsync

Sobrescribimos SaveChangesAsync en SolicitudesDbContext para que toda entidad que herede de EntidadAuditable reciba su fecha UTC automáticamente.

SolicitudesDbContext.cs

```
public override Task<int> SaveChangesAsync(CancellationToken cancellationToken = default)
{
    var entradas = ChangeTracker.Entries<EntidadAuditable>();

    foreach (var entrada in entradas)
    {
        if (entrada.State == EntityState.Added)
        {
            entrada.Entity.FechaCreacion = DateTime.UtcNow;
            entrada.Entity.CreadoPor = "sistema"; // Se conectará con el usuario actual en S13
        }
        else if (entrada.State == EntityState.Modified)
        {
            entrada.Entity.FechaModificacion = DateTime.UtcNow;
            entrada.Entity.ModificadoPor = "sistema";
        }
    }

    return base.SaveChangesAsync(cancellationToken);
}
```

Paso 3 — Consulta LINQ con Eager Loading (Include)

Escribimos la consulta que carga la solicitud junto a sus detalles y el usuario solicitante en un solo viaje al motor SQL.

ConsultasDemo.cs

```
// Eager Loading: genera un JOIN explícito en lugar de múltiples consultas separadas
var solicitudConDetalles = await db.Solicitudes
    .Include(s => s.Solicitante)
    .Include(s => s.Detalles)
    .Include(s => s.Aprobaciones)
    .FirstOrDefaultAsync(s => s.Id == 1);
```

Paso 4 — Optimización de solo lectura con AsNoTracking()

Para listar solicitudes en un reporte o endpoint GET, apagamos el Change Tracker para ahorrar memoria RAM y tiempo de CPU.

ConsultasDemo.cs

```
// AsNoTracking: hasta un 60% más rápido para consultas de solo lectura
var listadoLectura = await db.Solicitudes
    .AsNoTracking()
    .Include(s => s.Solicitante)
    .Where(s => s.Estado != EstadoSolicitud.Pagada)
    .OrderByDescending(s => s.FechaCreacion)
    .ToListAsync();
```

Paso 5 — Proyección con Select()

Proyectar con Select genera un SELECT en SQL únicamente con las columnas solicitadas, ignorando las que no se necesitan en el DTO.

ConsultasDemo.cs

```
var resumenDto = await db.Solicitudes
    .AsNoTracking()
    .Where(s => s.Estado == EstadoSolicitud.Enviada)
    .Select(s => new
    {
        s.Id,
        s.Motivo,
        s.MontoTotal,
        Solicitante = s.Solicitante != null ? s.Solicitante.NombreCompleto : "",
        TotalItems = s.Detalles.Count
    })
    .ToListAsync();
```

3. Retos adicionales

1. Escribe una prueba en consola que mida la diferencia de milisegundos entre una consulta con tracking y otra con AsNoTracking().
2. Usa el método `db.Solicitudes.AsNoTracking().Where(...).ToQueryString()` para imprimir la sentencia SQL exacta generada en la terminal.
3. Implementa una transacción atómica con `using var tx = await db.Database.BeginTransactionAsync()` para insertar una solicitud con sus detalles.

4. Crea una migración llamada MigracionSemilla que aplique los datos de HasData en la base de datos.

4. Entregables

- Base de datos con los roles y centros de costo iniciales poblados mediante migraciones de EF Core.
- Prueba funcional que demuestre que al agregar una entidad, FechaCreacion se llena sin asignación manual.
- Código C# con consultas LINQ optimizadas mediante Include(), AsNoTracking() y Select().
- Explicación técnica en un comentario de qué es el problema N+1 y cómo Include lo soluciona.

Técnico de Desarrollo Full Stack con C# y Angular · Sesión 4 · Docente: Edwin Barrientos · New Horizons